

A dark, blue-toned background image of a soccer stadium at night. The pitch is visible with players in motion, and the stands are filled with spectators. The lighting is dramatic, with bright spots on the field and the goal area.

Fi

CIÊNCIA DE DADOS

Python e R aplicados ao Futebol

Cauê Engelmann



CAUÊ ENGELMANN



São Paulo - SP



caue.engelmann@gmail.com



br.linkedin.com/in/caueengelmann

SOBRE MIM

- Cientista de Dados Sênior
- Bacharel em Ciência & Tecnologia
- MBA Artificial Intelligence & Machine Learning
- MBA TrendsInnovation Transformação Digital
- Instrutor de Deep Learning



AGENDA

01

Introdução ao R



02

Introdução ao Python



03

Introdução à
biblioteca Pandas



04

Introdução à
Visualização de Dados



AGENDA

01

Introdução ao R



02

Introdução ao Python



03

Introdução à
biblioteca Pandas



04

Introdução à
Visualização de Dados



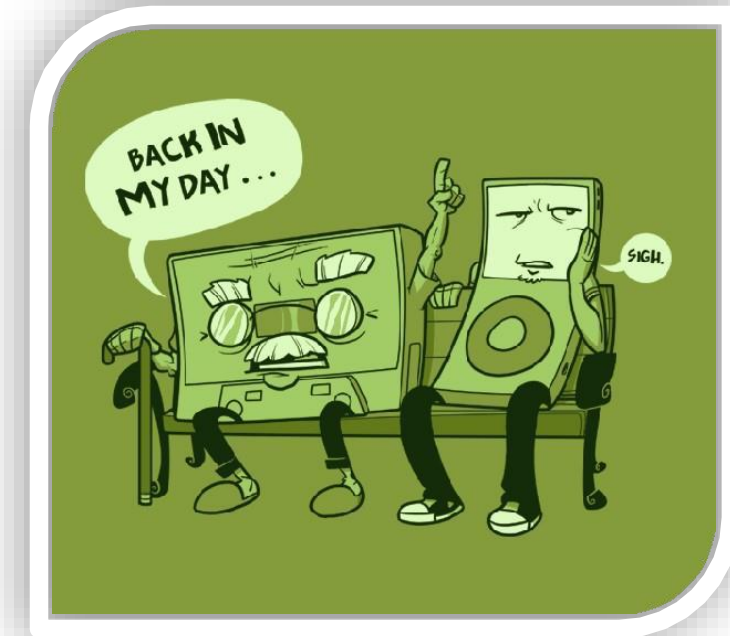
Principais características

- Software livre e Open Source
 - GNU General Public License
- Distribuição em Linux, Windows, Mac
 - Pode ser compilado para outras plataformas
- Linguagem interpretada, e não compilada
 - Ambiente exploratório sem compilação
- Tipagem Dinâmica
- Integração com bibliotecas de alto desempenho, compiladas em C, C++ e Fortran



Limitações importantes sobre o R

- Não há garantia de funcionamento e continuidade
 - Não há um contrato com uma empresa, como quando você compra uma licença de software
- Uma linguagem de mais de 40 anos
- Todos os dados ficam em memória
- Laços muito lentos (loop)
 - Velocidade de processamento muito lenta, se comparada a linguagens compiladas



O que é possível fazer com R

- Analytics



- Matemática e estatística do básico ao mais avançado
- Modelagem estatística, simulações de ambientes e otimização
- Aprendizado de Máquina – Machine Learning
- Integração e análise com Big Data

- Gráficos e Visualização

- Plotagens gráficas, simples a profissionais
- Gráficos dinâmicos
- Reprodução geográfica



E mais ...

O que é possível fazer com R

- Programação



- Criação de scripts complexos
- Documentação de processos
 - (já tentou reproduzir algo no excel?)

- Distribuição

- Possibilidade de criar e compartilhar pacotes
- Documentação
- Aplicações rodando em Back-End
- Front-end interativo



Instalar o R

- Página para Download:
<https://cran.r-project.org/>
 - Ou pelo R-Project:
www.r-project.org
 - Alternativo:
<https://cran.r-project.org/mirrors.html>
 - Microsoft R:
<https://mran.microsoft.com/download>



Instalar o RStudio e o RBuildTools

- Página para Download

<https://www.rstudio.com/products/rstudio/download/>



- R Build Tools

O Build Tools é instalado e configurado a partir do RStudio, ao se criar um projeto ou pelo link:

<https://cran.r-project.org/bin/windows/Rtools/Rtools33.exe>

Principais pacotes - Core Team (RStudio)



Popularidade

Python ultrapassa JavaScript no GitHub com crescimento de IA generativa, revela Octoverse 2024

GitHub relata salto de 98% em projetos de IA generativa, consolidando Python como linguagem mais popular da plataforma

Feb 2025	Feb 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.88%	+8.72%
2	3	▲		C++	11.37%	+0.84%
3	4	▲		Java	10.66%	+1.79%
4	2	▼		C	9.84%	-1.14%
5	5			C#	4.12%	-3.41%
				JavaScript	3.78%	+0.61%
				SQL	2.87%	+1.04%
				Go	2.26%	+0.53%
				Delphi/Object Pascal	2.18%	+0.78%
				Visual Basic	2.04%	+0.52%
				Fortran	1.75%	+0.35%
12	15	▲		Scratch	1.54%	+0.36%
13	18	▲		Rust	1.47%	+0.42%
14	10	▼		PHP	1.14%	-0.37%
15	21	▲		R	1.06%	+0.07%
16	13	▼		MATLAB	0.98%	-0.28%

TIOBE Index

AGENDA

01

Introdução ao R



02

Introdução ao Python



03

Introdução à
biblioteca Pandas



04

Introdução à
Visualização de Dados



Características

- Python é uma linguagem de altíssimo nível, orientada a objetos, de tipagem dinâmica e forte, interpretada e interativa.

Linguagem de Alto Nível

Orientada a Objetos

Tipagem Dinâmica e Forte

Interpretada

Interativa



Contém diversas estruturas de alto nível: listas, dicionários, data/hora, etc). É multiparadigma, ou seja, suporta programação funcional e modular além da orientação a objetos.

Características

- Python é uma linguagem de altíssimo nível, orientada a objetos, de tipagem dinâmica e forte, interpretada e interativa.

Linguagem de Alto Nível

Orientada a Objetos

Tipagem Dinâmica e Forte

Interpretada

Interativa



Paradigma de programação composto basicamente por 4 pilares: abstração, encapsulamento, herança e polimorfismo

Características

- Python é uma linguagem de altíssimo nível, orientada a objetos, de tipagem dinâmica e forte, interpretada e interativa.

Linguagem de Alto Nível

Orientada a Objetos

Tipagem Dinâmica e Forte

Interpretada

Interativa



Tipagem dinâmica: o tipo de uma variável é inferido pelo interpretador em tempo de execução

Tipagem forte: o interpretador verifica se as operações são válidas e não faz coerções automáticas entre tipos incompatíveis

Características

- Python é uma linguagem de altíssimo nível, orientada a objetos, de tipagem dinâmica e forte, interpretada e interativa.

Linguagem de Alto Nível

Orientada a Objetos

Tipagem Dinâmica e Forte

Interpretada

Interativa



Precisa de um interpretador para que um programa seja executado. Não tem um compilador que transforma uma linguagem “humana” em linguagem de máquina

Características

- Python é uma linguagem de altíssimo nível, orientada a objetos, de tipagem dinâmica e forte, interpretada e interativa.

Linguagem de Alto Nível

Orientada a Objetos

Tipagem Dinâmica e Forte

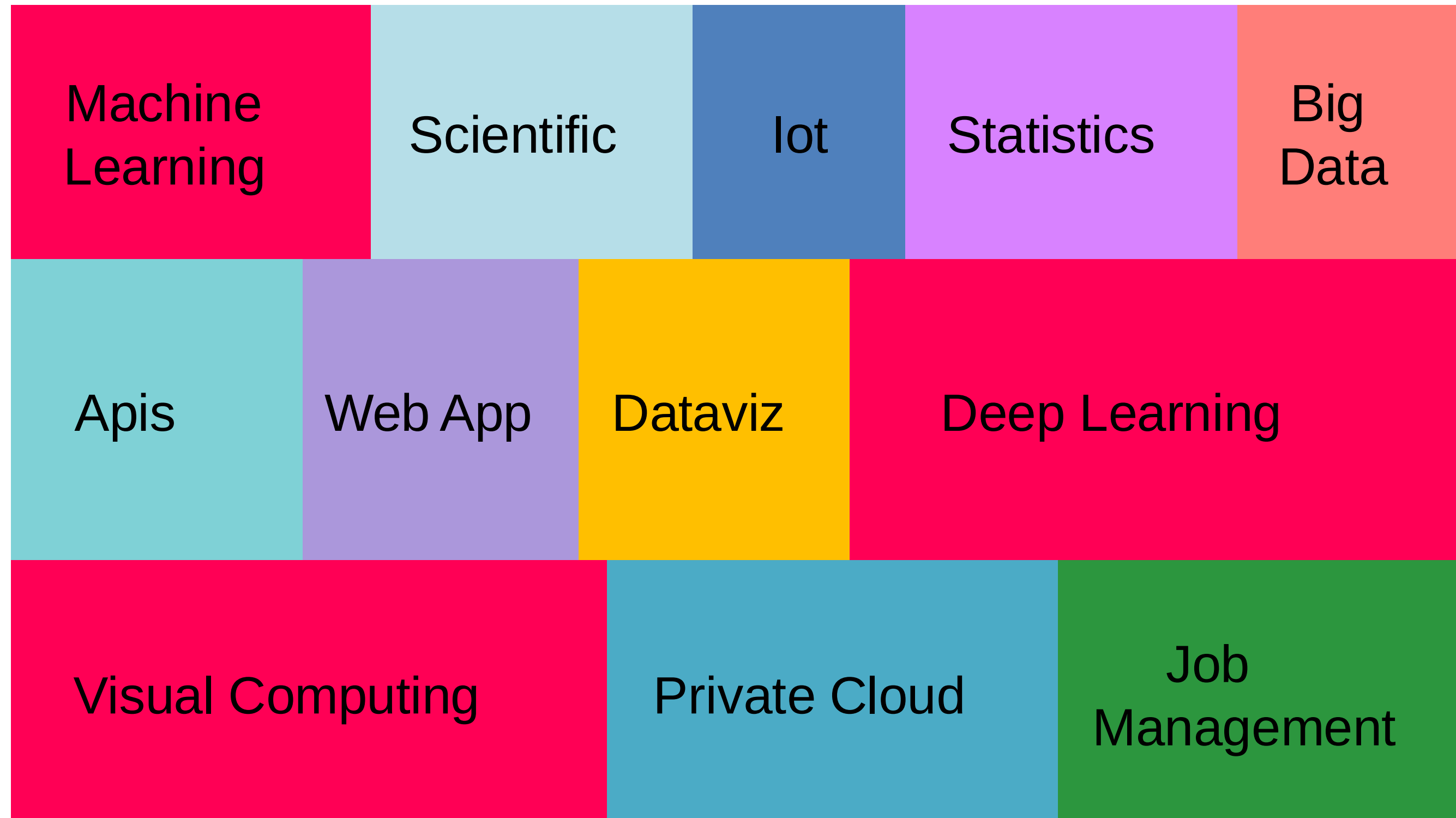
Interpretada

Interativa

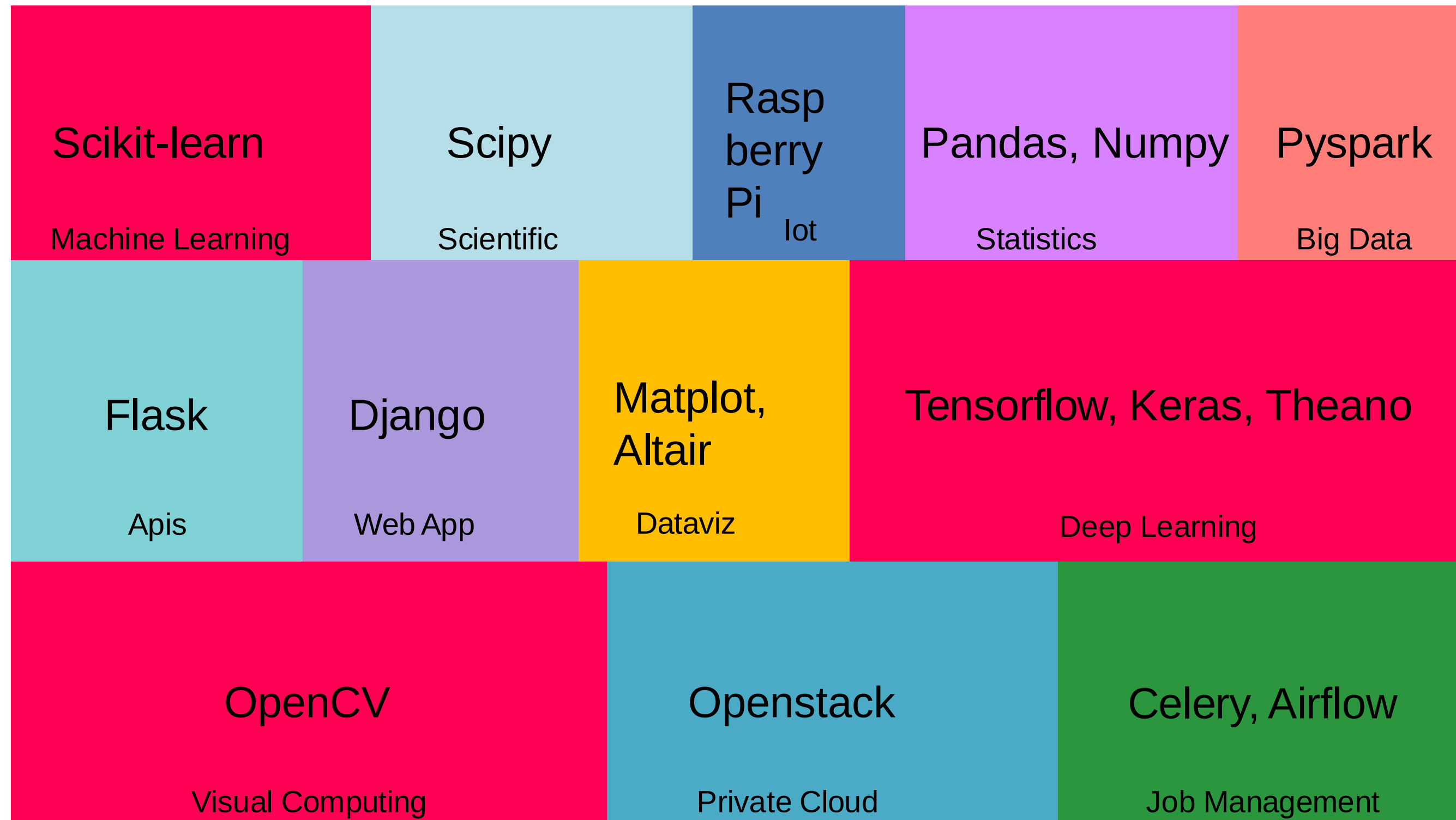


Execução de comandos
via shell

O que podemos fazer com Python?



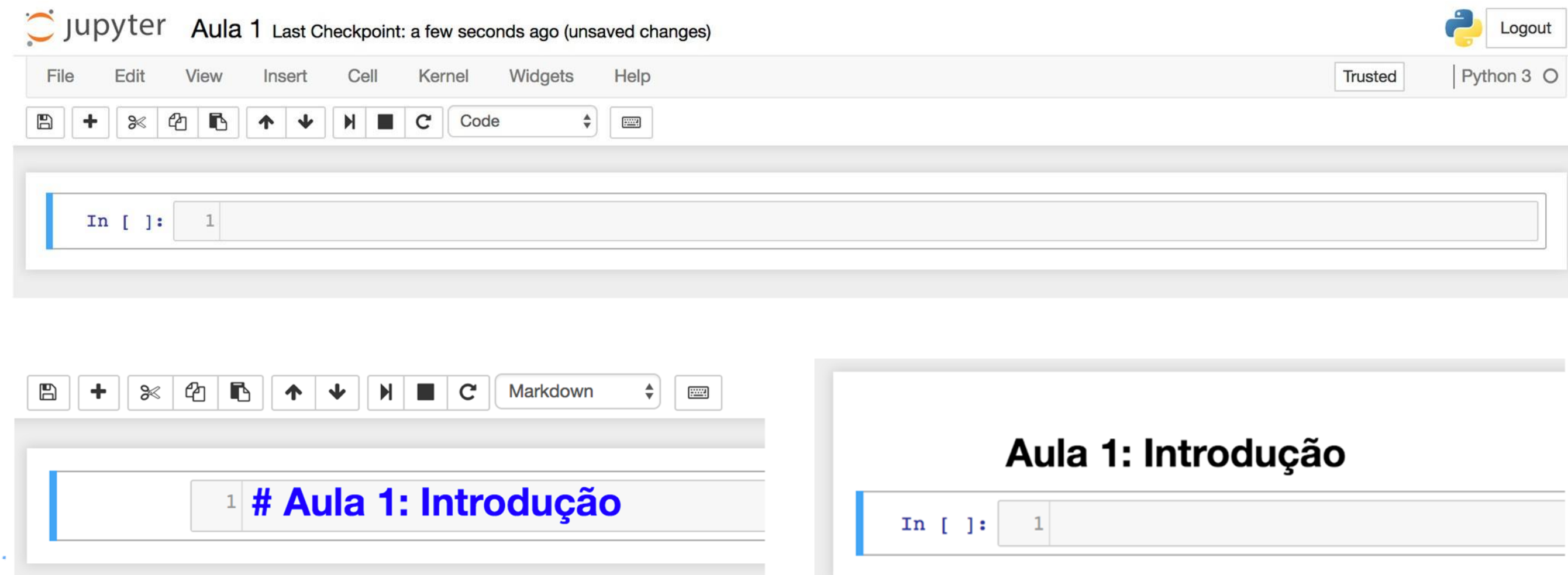
O que podemos fazer com Python?



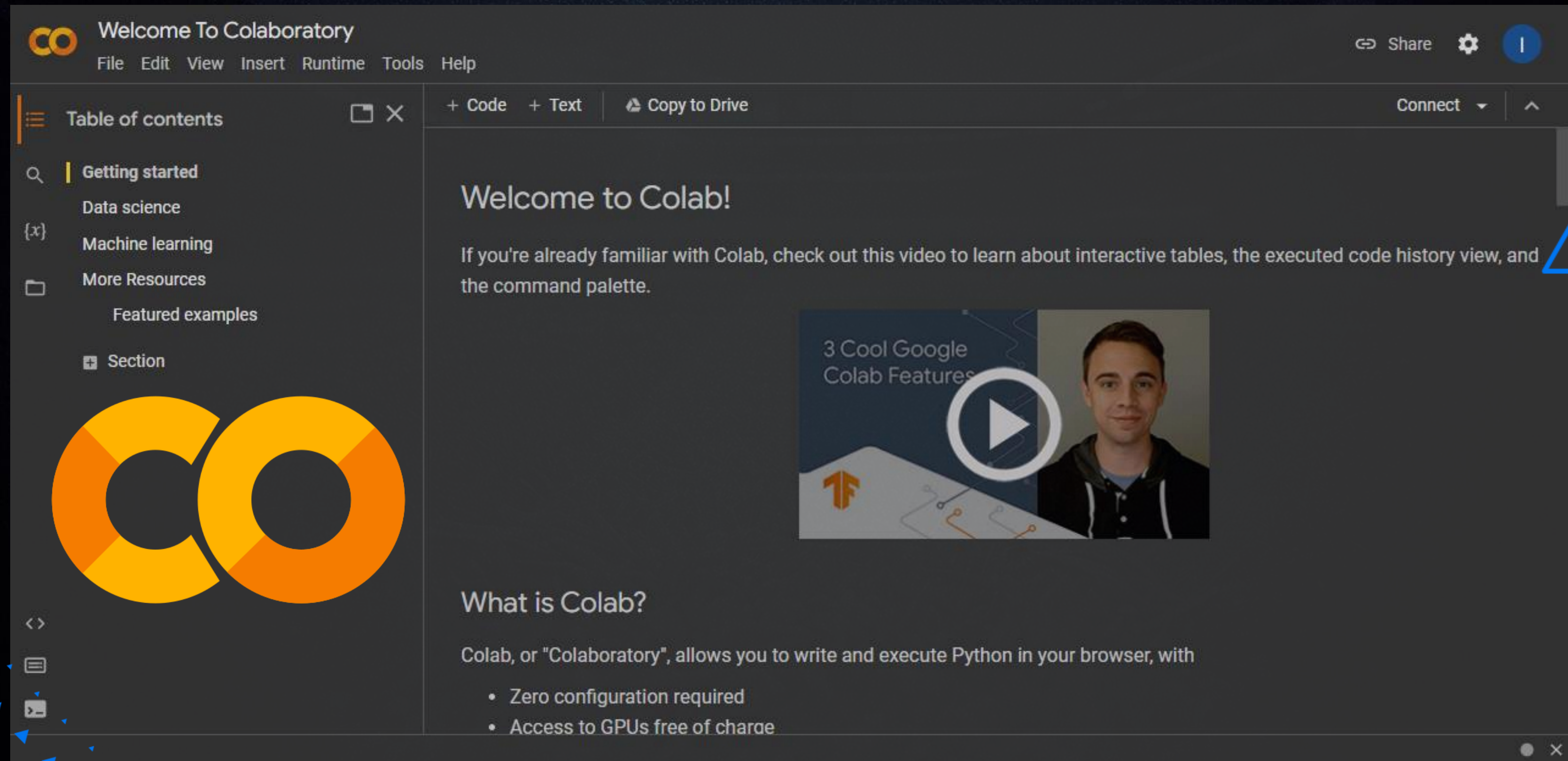
Jupyter notebook

Pode ser executado código Python ou Markdown.

Este último serve como forma de editar comentários acerca do código para ficar mais claro.



Google Colab



The screenshot displays the Google Colaboratory (Colab) interface. At the top, the title bar reads "Welcome To Colaboratory" with a menu bar containing "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". On the right of the title bar are "Share", "Settings", and a user profile icon. Below the title bar, there's a toolbar with "+ Code", "+ Text", and "Copy to Drive". The left sidebar features a "Table of contents" panel with a search icon, a list of categories ("Getting started", "Data science", "Machine learning", "More Resources", "Featured examples"), and a "Section" header. Below this is a large orange "CO" logo. The main content area has a heading "Welcome to Colab!" followed by a paragraph: "If you're already familiar with Colab, check out this video to learn about interactive tables, the executed code history view, and the command palette." Below the text is a video player showing a thumbnail with the title "3 Cool Google Colab Features" and a play button. At the bottom, a section titled "What is Colab?" explains that Colab allows writing and executing Python in the browser, with a bulleted list of features: "Zero configuration required" and "Access to GPUs free of charge".

Welcome To Colaboratory

File Edit View Insert Runtime Tools Help

Share Settings User

+ Code + Text Copy to Drive

Connect ^

Welcome to Colab!

If you're already familiar with Colab, check out this video to learn about interactive tables, the executed code history view, and the command palette.

3 Cool Google Colab Features

What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge

Sintaxe - Objetos

- Python é uma linguagem **orientada à objetos** e, sendo assim, cada um deles possuem **atributos** e **métodos** que podem ser acessados por ponto.

```
import numpy as np
```

```
x = np.ones(10)
```

```
y = x.sum() #método
```

```
y.argmax() # atributo - retorna o índice do maior valor no vetor
```

Sintaxe – Controle de Fluxo

- É muito comum em um programa que certos conjuntos de instruções sejam executados de forma condicional, em casos como validar entradas de dados, por exemplo.

```
if <condição>:  
    <bloco de código>  
elif <condição>:  
    <bloco de código>  
elif <condição>:  
    <bloco de código>  
else:  
    <bloco de código>
```

```
temp = int(input('Entre com a temperatura: '))  
if temp < 0:  
    print ('Congelando...')  
elif (0 <= temp <= 20):  
    print ('Frio')  
elif (21 <= temp <= 25):  
    print ('Normal')  
elif (26 <= temp <= 35):  
    print ('Quente')  
else:  
    print ('Muito quente!')
```


Sintaxe – Controle de Fluxo

- Se o bloco de código for composto de apenas uma linha, ele pode ser escrito após os dois pontos:

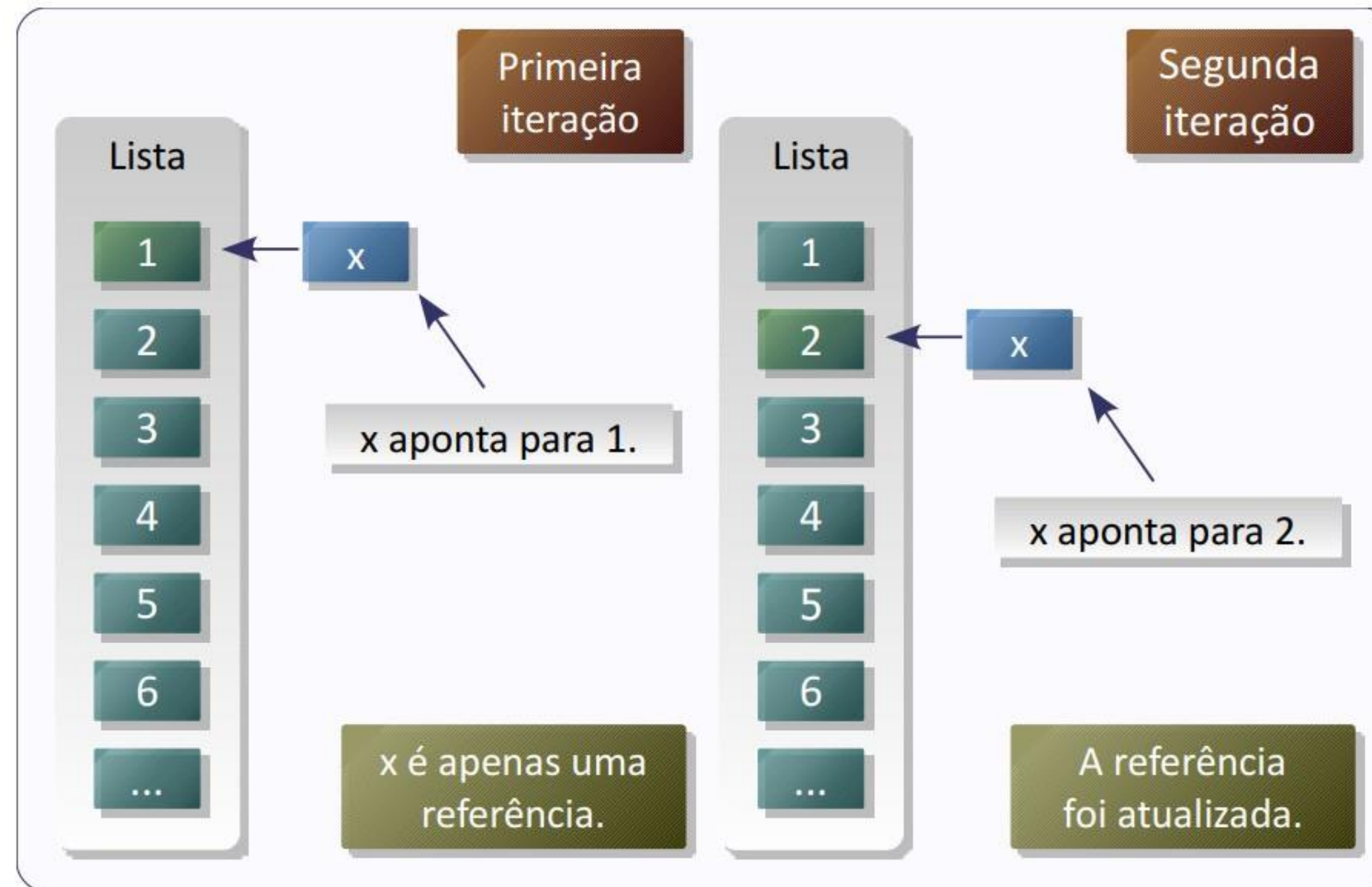
```
if temp < 0: print 'Congelando...'
```

- Pode-se usar a seguinte expressão também:
 - <variável> = <valor 1> if <condição> else <valor 2>

```
x = 'verdadeiro' if 1>0 else 'falso'
```

Sintaxe – Laços

- Laços (loops) são **estruturas de repetição**, geralmente usados para processar coleções de dados, tais como linhas de um arquivo ou registros de um banco de dados, que precisam ser processados por um mesmo bloco de código.



Sintaxe – Laços

Em Python, temos:

For Loop:

```
for <referência> in <sequência>:  
    <bloco de código>  
    continue  
    break  
else:  
    <bloco de código>
```

```
# Soma de 0 a 99  
s = 0  
for x in range(1, 100):  
    s = s + x  
print s
```

While Loop:

```
while <condição>:  
    <bloco de código>  
    continue  
    break  
else:  
    <bloco de código>
```

```
# Soma de 0 a 99  
s = 0  
x = 1  
  
while x < 100:  
    s = s + x  
    x = x + 1  
print s
```


While loop

```
parada = 10
inicio = 0

while parada > 0:
    print(parada)
    parada -= 1
    if parada == 3:
        break
    elif parada == 4:
        continue
    else:
        pass
else:
    print("final")
```

Break: sai do loop

Continue: move o cursor para o top do loop

Pass: não altera nada, continua normal.

Listas

- Listas são **coleções heterogêneas de objetos**, que podem ser de qualquer tipo, inclusive outras listas.
- As listas no Python são **mutáveis**, podendo ser alteradas a qualquer momento. Listas podem ser fatiadas da mesma forma que as *strings*, mas como as listas são mutáveis, é possível fazer atribuições a itens da lista.
- Sintaxe: `lista = [1,2,3]`

Listas

```
bandas = ['Queen', 'Led Zeppelin', 'Kansas', 'Europe', 'Guns N Roses']

for banda in bandas:
    print (banda)

bandas.append('Bon Jovi') #adiciona ao fim da lista
bandas.remove('Europe') #remove
bandas.sort() #ordena

print (bandas)
```

```
Queen
Led Zeppelin
Kansas
Europe
Guns N Roses
['Bon Jovi', 'Guns N Roses', 'Kansas', 'Led Zeppelin', 'Queen']
```


Listas

```
for i, banda in enumerate(bandas):  
    print (i + 1, '=>', banda)  
  
#filas e pilhas  
while bandas:  
    print ('Saiu', bandas.pop(0), ', faltam', len(bandas))
```

```
1 => Bon Jovi  
2 => Guns N Roses  
3 => Kansas  
4 => Led Zeppelin  
5 => Queen  
Saiu Bon Jovi , faltam 4  
Saiu Guns N Roses , faltam 3  
Saiu Kansas , faltam 2  
Saiu Led Zeppelin , faltam 1  
Saiu Queen , faltam 0
```

A função `enumerate()` retorna uma tupla de dois elementos a cada iteração: um número sequencial e um item da sequência correspondente.

Tuplas

- Semelhantes as listas, porém são **imutáveis**: não se pode acrescentar, apagar ou fazer atribuições aos itens.
- Sintaxe: `tupla = (a,b,c)`
 - Particularidade: tupla com apenas um elemento
 - `T1 = (1,)`
- Conversões
 - Lista em tupla: `tupla = tuple(lista)`
 - Tupla em lista: `lista = list(tupla)`

Dicionário

- Um dicionário é uma **lista de associações compostas por uma chave única e estruturas correspondentes**. Dicionários são mutáveis, tais como as listas.
- A chave precisa de um tipo imutável, já os itens podem ser de qualquer tipo.
 - Não há garantia de ordenação de chaves
- Sintaxe:
 - `dicionario = {'a': a, 'b': b, ..., 'z': z}`

Dicionário

Mustang

1964

marca Ford

modelo Mustang

Ano 2018

marca Ford

modelo Mustang

Ano 2018

cor preto

```
{ 'marca': 'Ford', 'modelo': 'Mustang', 'cor': 'preto' }
```

```
{ }
```

```
{ 'marca': 'Ford', 'modelo': 'Mustang', 'Ano': 1964 }
```

Funções

- Funções são blocos de código identificados por um nome, que podem receber parâmetros pré-determinados.
- Características:
 - Podem ou não retornar objeto
 - Aceitam *Doc Strings*
 - Tem *namespace* próprio
 - Aceitam parâmetros opcionais
- *Doc Strings* são documentações de uma estrutura

Funções

```
def func(parametro1, parametro2=padrao):  
    """Doc String  
    """  
    <bloco de código>  
    return valor
```

Funções - Exemplo

```
#implementação recursiva  
def fatorial(num):  
    if (num <= 1):  
        return 1  
    else:  
        return(num * fatorial(num - 1))
```

```
def numero_par(num):  
    resto = num % 2  
    if (resto == 0):  
        return True  
    else:  
        return False
```


AGENDA

01

Introdução ao R



02

Introdução ao Python



03

Introdução à
biblioteca Pandas



04

Introdução à
Visualização de Dados



Pandas

Pontes entre diferentes tipos de fontes de dados para utilização principalmente em preparação de dados para posteriormente ser aplicado com algoritmos de aprendizado de máquina.

EXCEL

JSON

TXT/CSV

Series

Estrutura de dados do tipo array, parecido com a estrutura do numpy.
Composto de índice e valores.
Pode ser nomeado a própria série e seu respectivo índice.

NOME	
ÍNDICE	VALORES
0	10000
1	20000
2	950000
NOME	

Índices

Os índices podem ser utilizados como se fossem uma chave de um dicionário. Permitindo tanto operações de leitura quanto de escrita.

```
import pandas as pd
from pandas import Series,
DataFrame
```

```
serie = Series([1,2,3,4,5,6])
serie
```

```
pib_paises = Series([10000000, 8000000,
7000000, 6000000, 5000000], index = ["China",
"Estados Unidos", "Canadá", "Rússia", "Brasil"] )
pib_paises
```

```
China          10000000
Estados Unidos  8000000
Canadá         7000000
Rússia         6000000
Brasil         5000000
dtype: int64
```

```
pib_paises["Brasil"]
5000000
```


Filtros

Filtrar pelo nome da série irá processar todas as colunas e linhas. Quando é realizado somente na notação de dicionário, será considerando somente a chave (coluna) em questão. Os nomes de índices também podem ser especificados a partir de outra lista.

```
pib_países[pib_países > 7000000]
```

```
China          100000000  
Estados Unidos  80000000
```

```
dtype: int64
```

```
"Argentina" in pib_países
```

```
False
```

```
pib_países_dict = pib_países.to_dict()
```

```
pib_países_dict
```

```
{'Brasil': 5000000,  
 'Canadá': 7000000,  
 'China': 10000000,  
 'Estados Unidos': 8000000,  
 'Rússia': 6000000}
```

```
lista_países = ["China", "Estados  
Unidos", "Canadá", "Rússia", "Brasil",  
"Peru"]
```

```
série_2 = Series(pib_países_dict, index  
= lista_países)
```

```
série_2
```

```
China          100000000.0  
Estados Unidos  80000000.0
```

```
Canadá         70000000.0
```

```
Rússia         60000000.0
```

```
Brasil         50000000.0
```

```
Peru           NaN
```

```
dtype: float64
```

Validações e nomes

O Pandas possui algumas funções de verificação de valores nulo, que busca todos os dados de todas as colunas.

```
pd.isnull(serie_2)
```

China	False
Estados Unidos	False
Canadá	False
Rússia	False
Brasil	False
Peru	True

```
dtype: bool
```

```
pd.notnull(serie_2)
```

China	True
Estados Unidos	True
Canadá	True
Rússia	True
Brasil	True
Peru	False

```
dtype: bool
```

Configuração de nome de índice e da série

```
serie_2.name="Produto interno bruto"  
serie_2.index.name="País"  
serie_2
```

País	
China	10000000.0
Estados Unidos	8000000.0
Canadá	7000000.0
Rússia	6000000.0
Brasil	5000000.0
Peru	NaN

Name: Produto interno bruto, dtype: float64

Dataframes

Estrutura de dados semelhante a uma tabela, composta por colunas e linhas.
Incorpora uma coleção de series.

PIB		PAÍSES		POPULAÇÃO	
ÍNDICE	VALORES	ÍNDICE	VALORES	ÍNDICE	VALORES
0	10000	0	EUA	0	860990
1	20000	1	PERU	1	781000
2	950000	2	CHINA	2	1000288

Inspeção

Por colunas e por seu conteúdo, tanto na forma de dicionário quanto na direta.

```
breaking_bad_temporada_4.columns  
Index(['Nº', 'Nº.1', 'Título', 'Dirigido por:',  
      'Escrito por:', 'Audiência', 'Exibição Original'],  
      dtype='object')
```

```
breaking_bad_temporada_4.Título
```

1	Box Cutter	Forma direta
2	Thirty-Eight Snub	
3	Open House	
4	Bullet Points	
5	Shotgun	
6	Cornered	
7	Problem Dog	
8	Hermanos	
9	Bug	
10	Salud	
11	Crawl Space	
12	End Times	
13	Face Off	

Name: Título, dtype: object

```
breaking_bad_temporada_4["Dirigido por:"]
```

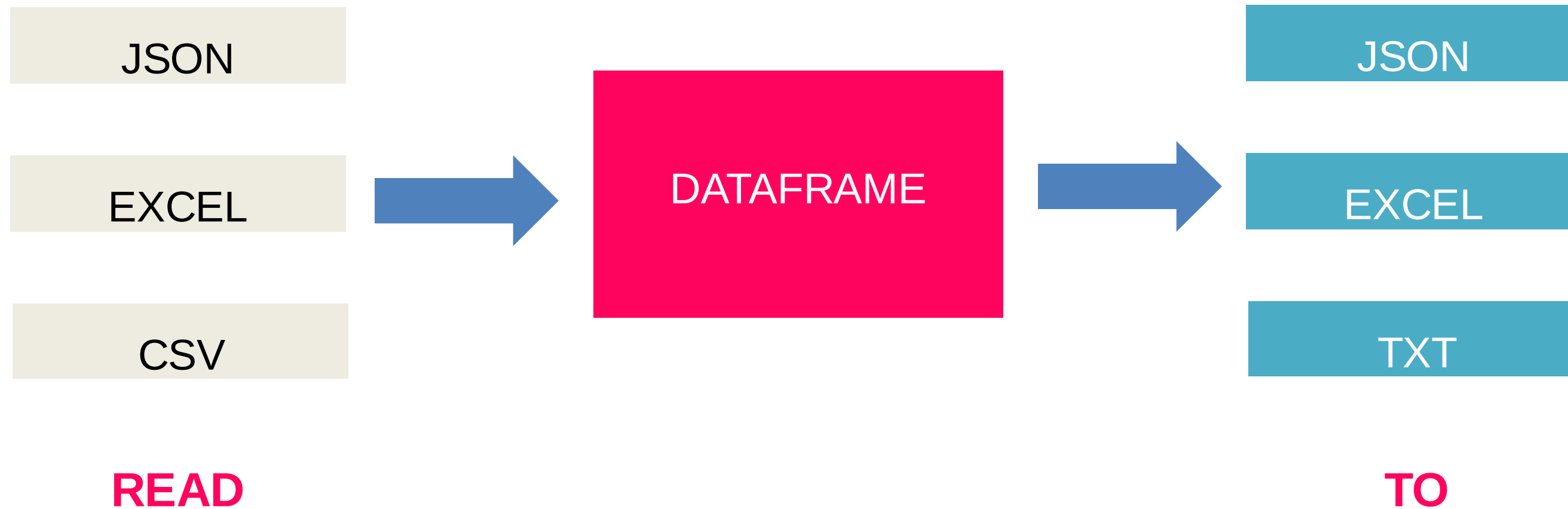
Notação dicionário

1	Adam Bernstein
2	Michelle MacLaren
3	David Slade
4	Colin Bucksey
5	Michelle MacLaren
6	Michael Slovis
7	Peter Gould
8	Johan Renck
9	Terry McDonough
10	Michelle MacLaren
11	Scott Winant
12	Vince Gilligan
13	Vince Gilligan

Name: Dirigido por:, dtype: object

Dados externos

Importação e exportação de arquivos para trabalhar como [dataframes](#) utiliza diversos formatos, dentre eles, JSON, Excel e CSV (Comma Separated Values).



Formato CSV

O parâmetro “header” (linha de título), se for None é ignorado. O padrão é “infer” que identifica automaticamente. Parâmetro “sep” é o separador de campo.

```
data_frame_csv = pd.read_csv("exemplo.csv")
data_frame_csv
```

	Ano;Livros;Jogos;Cadernos
0	2015;111;53;32
1	2016;224;455;43
2	2017;434;567;65
3	2018;564;2333;77

```
data_frame_csv = pd.read_csv("exemplo.csv", sep=";")
data_frame_csv
```

	Ano	Livros	Jogos	Cadernos
0	2015	111	53	32
1	2016	224	455	43
2	2017	434	567	65
3	2018	564	2333	77

Formato CSV

O método **describe** retorna uma descrição de todas as colunas de um dataframe, incluindo quantidade, média, desvio padrão e quartis, dentre outros. Utilizando a operação “to_csv”, armazenaremos as informações descritivas em um novo arquivo CSV (“exemplo_desc.csv”).

```
data_frame_desc = data_frame_csv.describe()
data_frame_desc
```

	Ano	Livros	Jogos	Cadernos
count	3.0	3.000000	3.000000	3.000000
mean	2017.0	407.333333	1118.333333	61.666667
std	1.0	171.561456	1053.421726	17.243356
min	2016.0	224.000000	455.000000	43.000000
25%	2016.5	329.000000	511.000000	54.000000
50%	2017.0	434.000000	567.000000	65.000000
75%	2017.5	499.000000	1450.000000	71.000000
max	2018.0	564.000000	2333.000000	77.000000

```
data_frame_desc.to_csv("exemplo_desc.csv")
```

Formato JSON

```
[
  {
    "state": "Florida", "shortname": "FL",
    "info": {
      "owner": "Edmund Kirsh"
    },
    "cars": [
      {
        "name": "Tesla", "model": "Model X"
      },
      {
        "name": "Ferrari", "model": "F50"
      }
    ]
  },
  {
    "state": "Ohio", "shortname": "OH",
    "info": {
      "owner": "Antonio Valdespino"
    },
    "cars": [
      {
        "name": "Fiat", "model": "500"
      }
    ]
  }
]
```

O formato JSON (Javascript Object Notation) é compatível com dicionários.

Com o Pandas é necessário fazer algumas normalizações para ter o dado visível como uma tabela. Neste caso trabalharemos com o método **normalization**.

O pandas possui métodos nativos para lidar com JSON.

Formato JSON

Os dados estão agrupados. Nesse caso podemos utilizar a normalização.

```
import json

data = json.load(open("exemplo.json"))
print(data)

[{'state': 'Florida', 'shortname': 'FL', 'info': {'owner': 'Edmund Kirsh'}, 'cars': [{'name': 'Tesla', 'model': 'Model X'}, {'name': 'Ferrari', 'model': 'F50'}]}, {'state': 'Ohio', 'shortname': 'OH', 'info': {'owner': 'Antonio Valdespino'}, 'cars': [{'name': 'Fiat', 'model': '500'}]}]
```

```
data_frame_json = pd.read_json("exemplo.json")
data_frame_json
```

	cars	info	shortname	state
0	[{'name': 'Tesla', 'model': 'Model X'}, {'name': 'Ferrari', 'model': 'F50'}]	{'owner': 'Edmund Kirsh'}	FL	Florida
1	[{'name': 'Fiat', 'model': '500'}]	{'owner': 'Antonio Valdespino'}	OH	Ohio



Formato Excel

Abrir arquivos no formato Excel, por padrão, utiliza a primeira guia (*sheet*) existente.

```
data_frame_excel = pd.read_excel("exemplo.xlsx")
data_frame_excel
```

	Filme	Avaliação	Custo
0	Avengers	3	210000
1	Avengers 2	4	250000
2	Avengers 3	4	400000
3	Iron Man	2	200000
4	Iron Man 2	2	200000
5	Iron Man 3	1	240000
6	Thor	4	240000
7	Thor 2	4	200000
8	Thor 3	5	230000
9	Captain America	5	180000
10	Captain America 2	5	200000
11	Captain America 2	3	280000

Formato Excel

Neste caso, obtém os dados da guia especificada, a guia “Fonte_1”.

```
data_frame_excel = pd.read_excel("exemplo.xlsx", sheetname="Fonte_1")
data_frame_excel
```

	Filme	Avaliação	Custo
0	Avengers	3	210000
1	Avengers 2	4	250000
2	Avengers 3	4	400000
3	Iron Man	2	200000
4	Iron Man 2	2	200000
5	Iron Man 3	1	240000
6	Thor	4	240000
7	Thor 2	4	200000
8	Thor 3	5	230000
9	Captain America	5	180000
10	Captain America 2	5	200000
11	Captain America 2	3	280000

AGENDA

01

Introdução ao R



02

Introdução ao Python



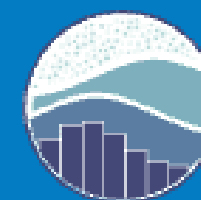
03

Introdução à
biblioteca Pandas



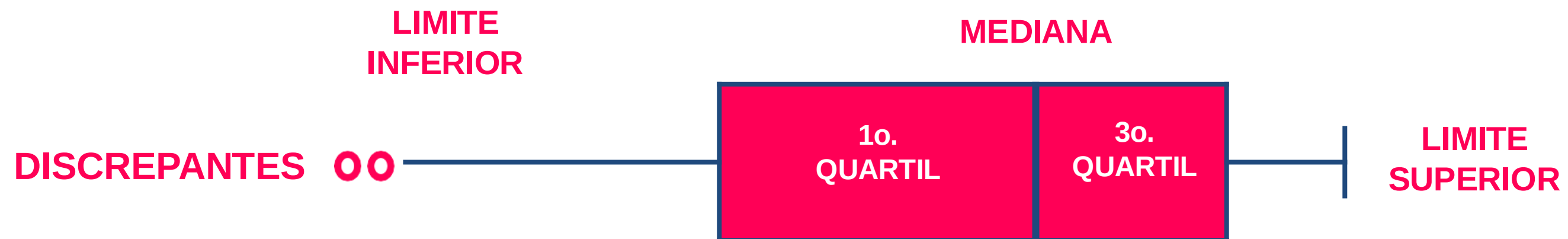
04

Introdução à
Visualização de Dados



Gráficos do tipo caixa (*boxplot*)

São tipos de gráficos que demonstram a variação de valores. Essencialmente úteis para identificar outliers de um conjunto de dados.

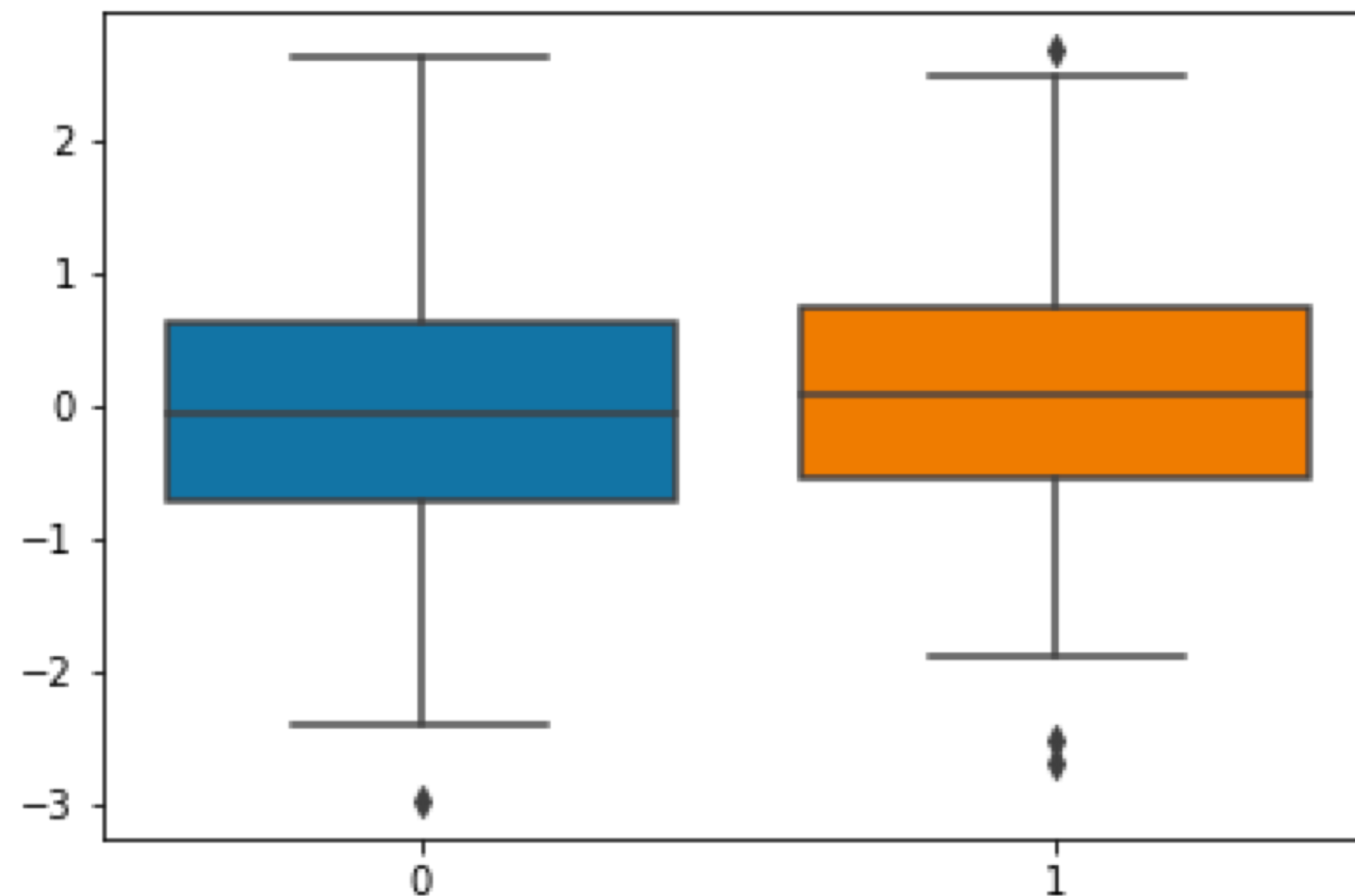


Gráficos do tipo caixa (*boxplot*)

Por padrão, os pontos discrepantes são 1,5 vezes entre o primeiro e o terceiro quartil.

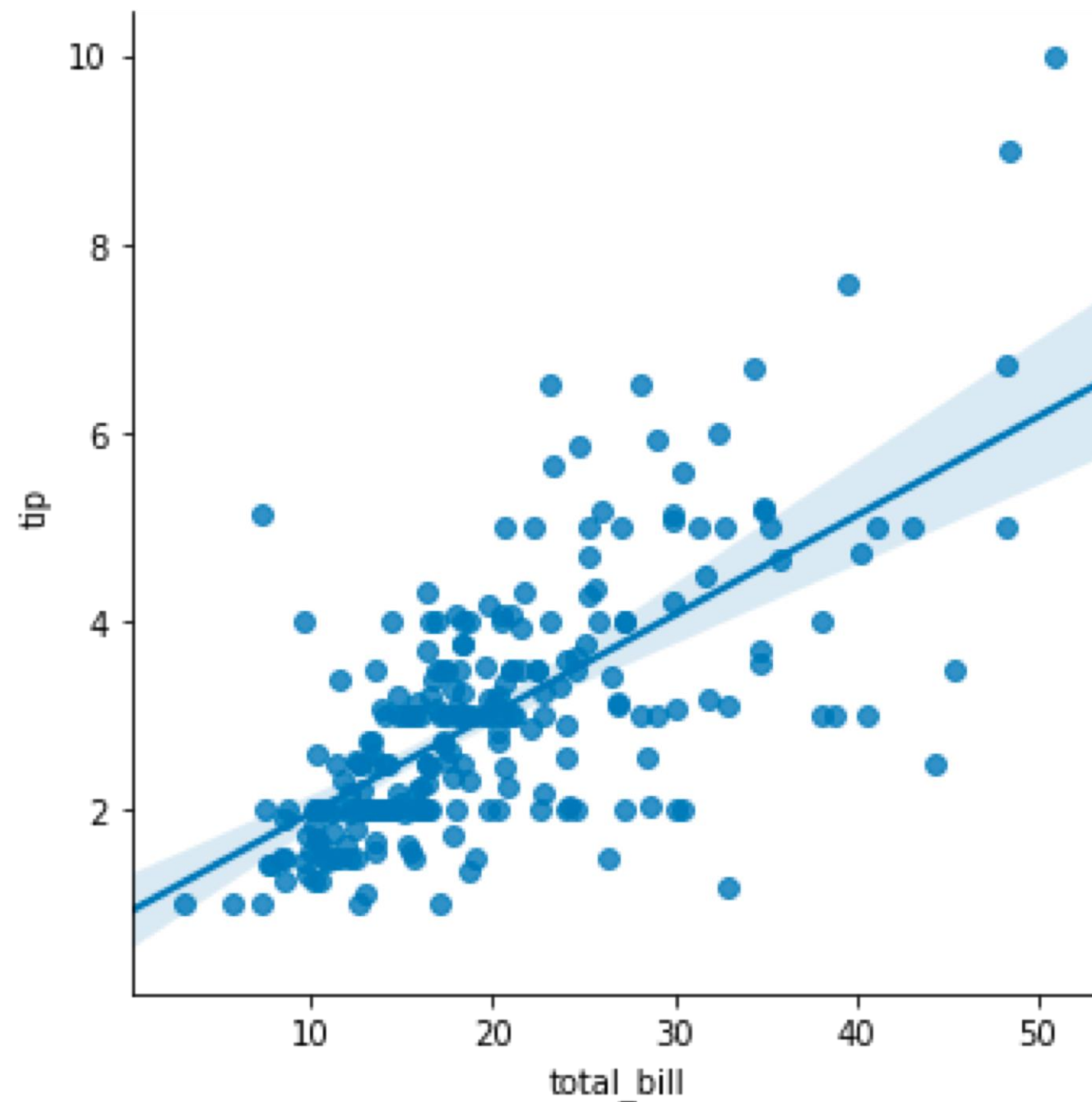
```
data_set_1 = np.random.randn(200)
data_set_2 = np.random.randn(200)

sns.boxplot(data=[data_set_1, data_set_2])
```



Gráficos de regressão e *scatterplot*

Gráficos que utilizam 2 variáveis, exibindo a correlação entre elas e sua tendência.



**RELAÇÃO ENTRE
VALOR DA GORJETA E
O TOTAL DA CONTA**

Gráficos de regressão e *scatterplot*

O Seaborn tem alguns conjuntos de dados para testes. O “tips” é uma tabela que relaciona dados de consumo de um restaurante.

```
gorjetas = sns.load_dataset("tips")
gorjetas.head(5)
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
sns.lmplot("total_bill", "tip", gorjetas)
```


Gráficos de linha

Vamos considerar o seguinte conjunto de dados.

```
data_frame_2 = pd.read_excel("bilheteria.xlsx")
data_frame_2
```

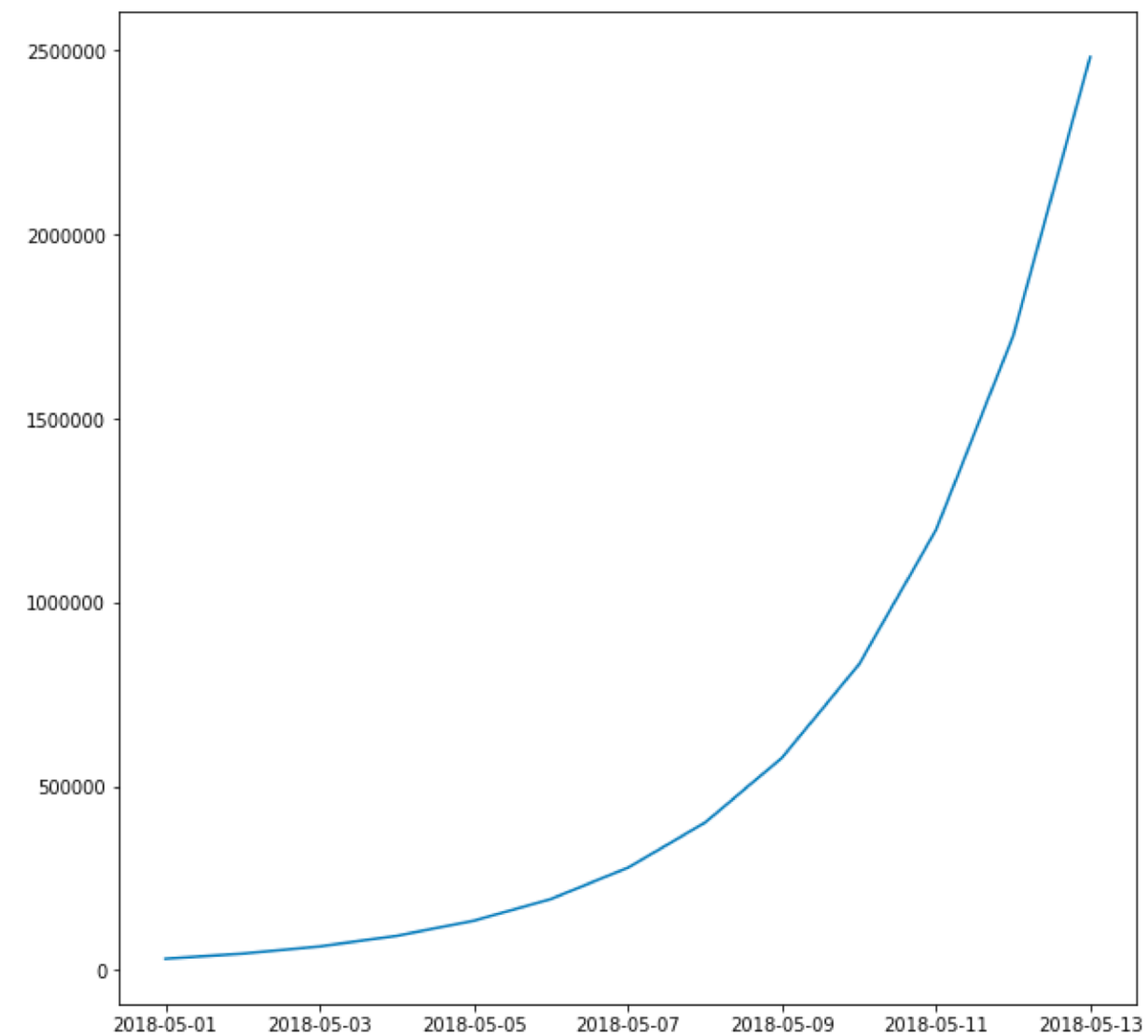
	Data	Bilheteria
0	2018-05-01	3.123100e+04
1	2018-05-02	4.497264e+04
2	2018-05-03	6.476060e+04
3	2018-05-04	9.325527e+04
4	2018-05-05	1.342876e+05
5	2018-05-06	1.933741e+05
6	2018-05-07	2.784587e+05
7	2018-05-08	4.009806e+05



Gráficos de linha

Podemos utilizar o **plot** do Matplotlib para este tipo de gráfico, que necessita das 2 dimensões do gráfico (eixo y e eixo x).

```
fig, ax = plt.subplots()
fig.set_size_inches(18, 8)
plt.plot(data_frame_2["Data"], data_frame_2["Bilheteria"])
```



Gráficos de barra

Há também o gráfico **barplot** do Seaborn com mais opções de configurações. Vamos considerar o conjunto de dados de **car_crashes** a seguir.

```
Batidas = sns.Load_dataset("car_crashes").Sort_values("total", ascending=False)
batidas_seg = batidas[(batidas["abbrev"] == 'SC') | (batidas["abbrev"] == 'ND') |
(batidas["abbrev"] == 'AR')]
batidas_seg.Head(5)
```

total	speeding	alcohol	not_distracted	no_previous	ins_premium	ins_losses	abbrev
23.9	9.082	9.799	22.944	19.359	858.97	116.29	SC
23.9	5.497	10.038	23.661	20.554	688.75	109.72	ND
22.4	4.032	5.824	21.056	21.280	827.34	142.39	AR



Gráficos de barra

O **barplot** pode ser configurado na forma de empilhamento. Neste caso precisamos adicionar mais uma série com a mesma variável em **y**.

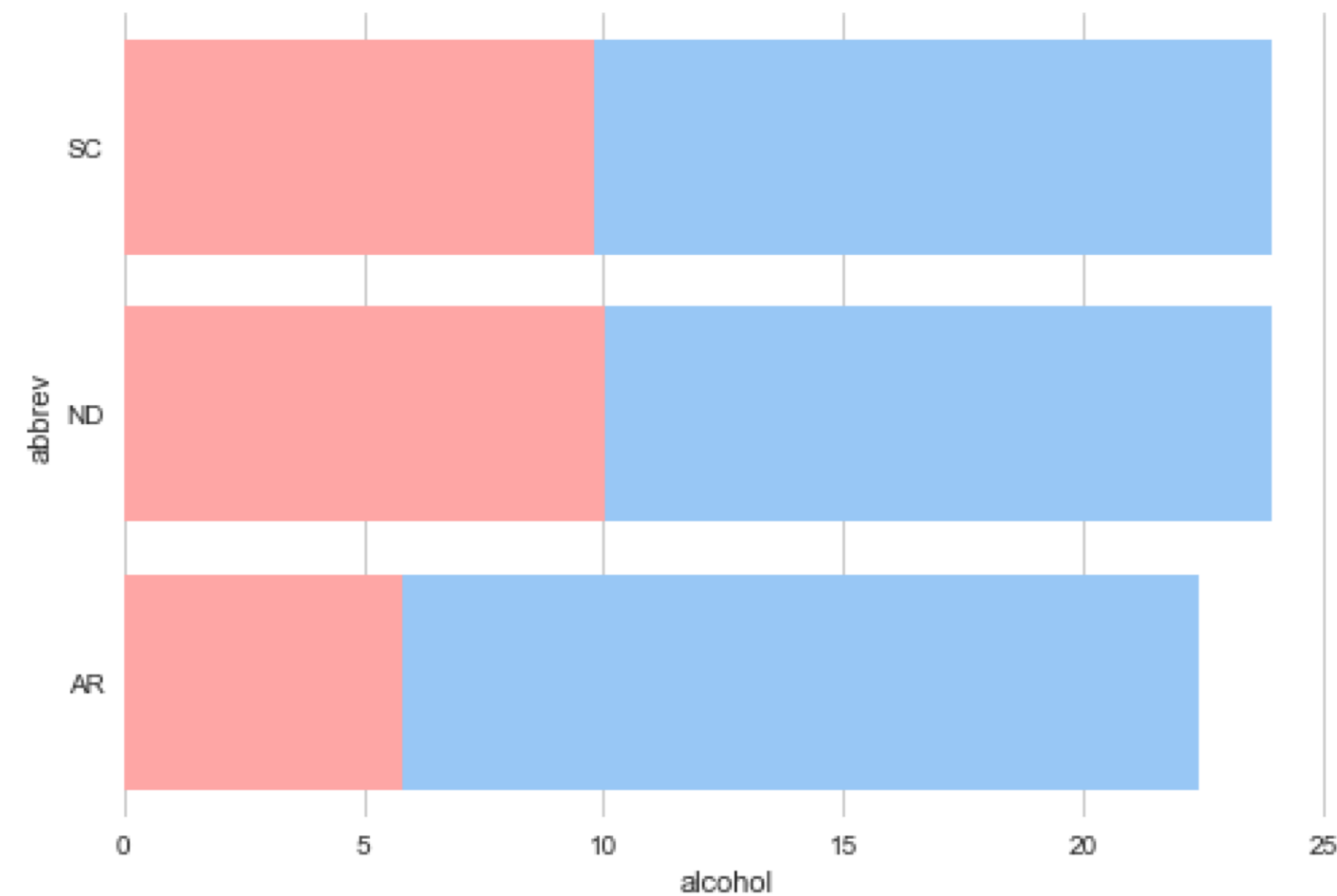
```
sns.set(style="whitegrid")
sns.set_color_codes("pastel")

sns.barplot(x="total", y="abbrev", data=batidas_seg, label="Total", color="b")
sns.barplot(x="alcohol", y="abbrev", data=batidas_seg, label="Alcohol-involved", color="r")

ax.legend(ncol=2, loc="lower right", frameon=True)
ax.set(xlim=(0, 24), ylabel="", xlabel="Automobile collisions per billion miles")
sns.despine(left=True, bottom=True)
```


Gráficos de barra

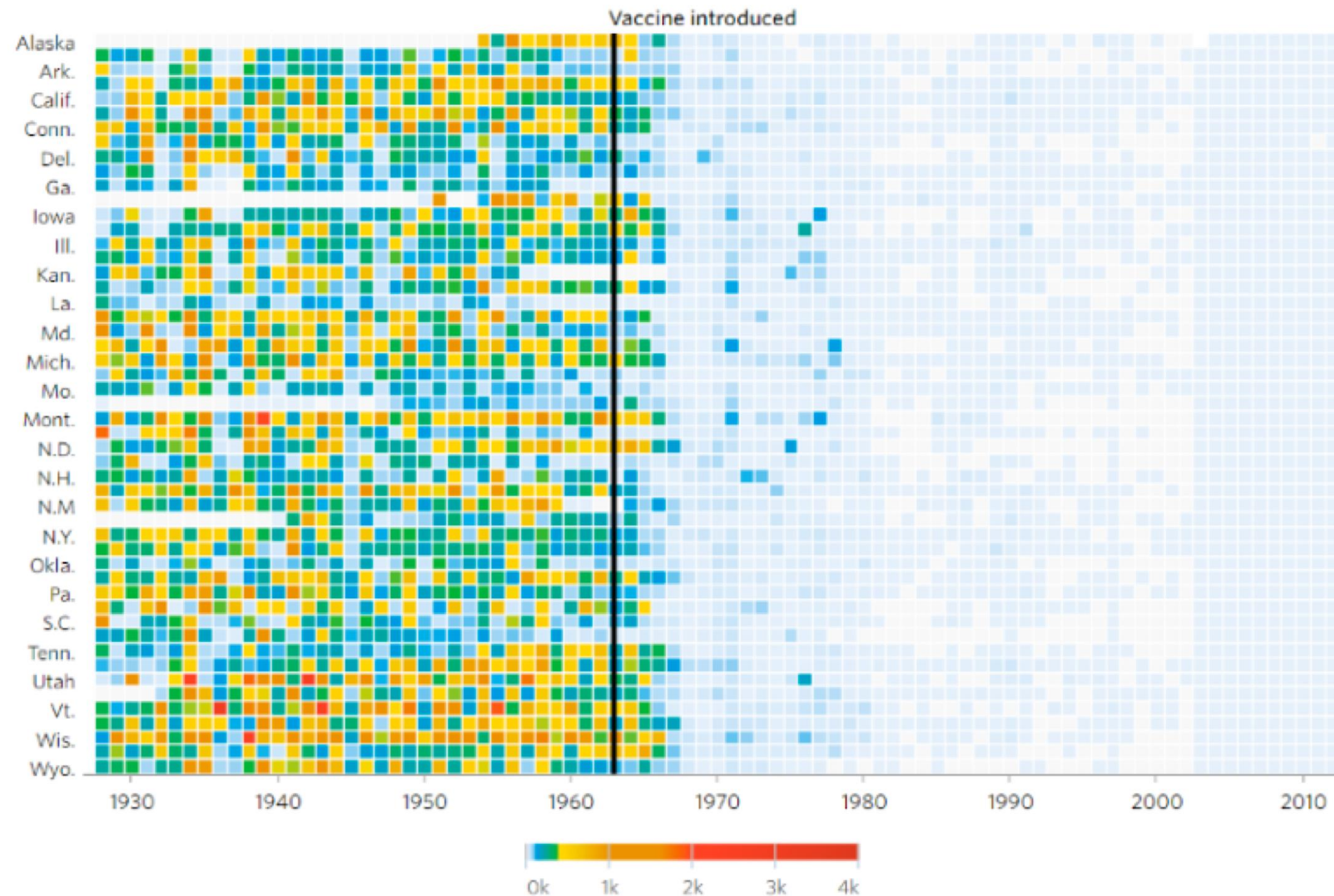
Ao final temos um gráfico de barra indicando, em vermelho, o peso das batidas de automóveis provocadas pelo uso de álcool nos três diferentes estados dos EUA.



Mapa de calor

Visualização que permite focar em concentrações de determinado atributos.

Measles



Mapa de calor

Utilizando o conjunto de dados **flights** vamos explorar alguns padrões.

```
voos = sns.load_dataset("flights")  
voos.head(5)
```

	year	month	passengers
0	1949	January	112
1	1949	February	118
2	1949	March	132
3	1949	April	129
4	1949	May	121

Mapa de calor

Agora vamos pivotar para o tipo de dataframe ficar adequado ao uso do mapa de calor.

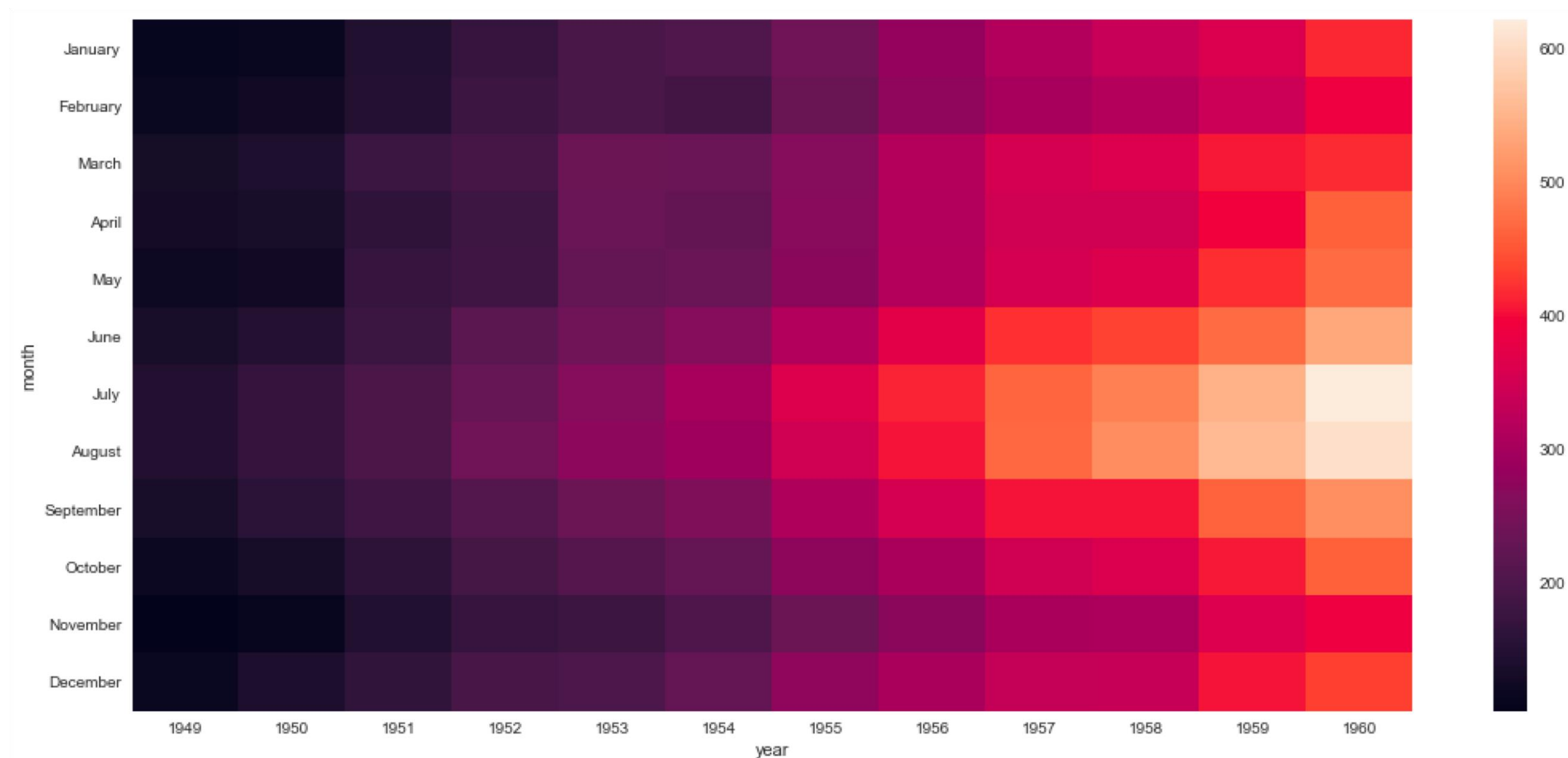
```
voos_pivoted = voos.pivot("month", "year", "passengers")  
voos_pivoted.head(5)
```

year	1949	1950	1951	1952	1953	1954	1955	1956	1957	1958	1959	1960
month												
January	112	115	145	171	196	204	242	284	315	340	360	417
February	118	126	150	180	196	188	233	277	301	318	342	391
March	132	141	178	193	236	235	267	317	356	362	406	419
April	129	135	163	181	235	227	269	313	348	348	396	461
May	121	125	172	183	229	234	270	318	355	363	420	472

Mapa de calor

Os pontos mais claros indicam maior agrupamento e coincidem com a popularização da avião no verão dos EUA.

```
fig, ax = plt.subplots()
fig.set_size_inches(18, 8)
sns.heatmap(voos_pivoted)
```





Dúvidas?

CAUÊ ENGELMANN



caue.engelmann@gmail.com



br.linkedin.com/in/caueengelmann





TRANSFORMANDO SONHOS EM CARREIRAS



FUTEBOLINTERATIVOBR



FUTEBOL INTERATIVO



FUTEBOLINTBR